MSCC on Raspberry Pi — install in plain English

> **Preferred single guide + current `.deb` kit: `../../installers/rpi/INSTALL.md`** and **`installers/rpi/`**. This file remains as the long-form servers install narrative.

For: Raspberry Pi **4 or 5**, **64-bit Raspberry Pi OS** (desktop recommended). **Goal:** Install MSCC, configure it, and start/stop the servers — mostly point-and-click after the package install.

Packages (download from the Releases page, https://github.com/w4mmp-ron/mscc-station/releases):

Package	Filename	
MSCC (everything for the radio)	`mscc_<version>_arm64.deb`	**required**
MSCC Avalonia client	`mscc-avalonia_<version>_arm64.deb`	optional (operate from the Pi itself)

The current version numbers are in the **MSCC Software Versions Addendum**.

Since mscc 1.0.79 (2026-11-10) the `mscc` package holds what used to be separate packages: PortAudio (`mscc-portaudio`), MSCC Init (`mscc-init`), MSCC Volume (`mscc-volume`) and the PSoC radio firmware (`mscc-firmware`), plus the Black Pill firmware. Installing it removes those old packages by itself.

Big picture (read this once)

Where	What runs
Raspberry Pi	Three **servers** (always-on radio + audio stack) and setup tools
Your PC	**MSCC client** (screen, knobs, spectrum) — this is how you operate

The Pi does **not** need the MSCC client window. That is normal: **headless servers on the Pi, GUI on the PC.**

```
PC (MSCC client)  ← network →  Pi (ms-sdr + sdrcore-recv + sdrcore-trans)  ← USB → Proficio
```

Two audio “worlds” on the Pi:

Path	What you hear / use	Devices
Operator (phones)	Headphones / mic at the Pi	Real sound card — see **Operator audio hardware** below. **Any sample rate is OK** (servers adapt).
Digital (WSJT etc.)	Digi app audio on the Pi	**VirtualA** / **VirtualB** (software cables — not a physical cable)

Operator audio hardware (important on Pi 4)

The Proficio radio always uses **USB audio** for I/Q. Operator phones and mic should preferably use a **different kind of path** so the Pi is not running two busy USB sound devices at once.

Operator phones / mic	Pi 4	Pi 5	Notes
I²S audio HAT (e.g. AudioInjector, Codec Zero, HiFiBerry with ADC+DAC)	**Recommended**	Recommended	Proven, stable approach
Pi headphone jack / onboard audio	OK for listen	OK	Limited mic options
Digital only (VirtualA / VirtualB)	**Recommended** for digi	Recommended	No second physical sound card
Second USB headset or USB sound dongle (with Proficio also on USB)	**Not recommended**	Often OK	On Pi 4, under load this can interrupt USB audio; spectrum and phones may freeze while servers still look "running"

In short: On **Raspberry Pi 4**, choose an **I²S HAT** or the **Pi jack** for operator audio, or work digi via **VirtualA/B**. Reserve a **USB headset plus Proficio** mainly for **Pi 5**, or accept that dual USB audio is less reliable on Pi 4.

This is a **supported configuration choice**, not a defective install: Pi 4 is fully supported when operator audio is not a second USB sound device.

What you need

1. Pi 4/5 with Raspberry Pi OS **desktop** (menu + Terminal).
2. The `.deb` files from `installers/rpi/` on the Pi (USB stick, Downloads folder, etc.).
3. Network on the Pi the first time (so `apt` can pull libraries).
4. Multus / Proficio powered on when you configure (recommended).
5. PC and Pi on the same network (or as your site requires) for the MSCC client.
6. **Operator audio plan** (see table above) — especially on **Pi 4**, plan an **I²S HAT** or digi path rather than a USB headset next to the Proficio.

Important: use Terminal from the Desktop

Package install still needs a short command line. Do this on the Pi:

7. Log into the **Raspberry Pi desktop** (monitor or the usual desktop session).
8. Open **Terminal** from the desktop menu.
9. Run the install commands **in that Terminal**.

You only need enough CLI to type a few lines and enter your password. After install, use the menu:

MSCC → MSCC Init, **MSCC Start**, **MSCC Stop**, **MSCC Status**

(Log out/in once if the **MSCC** category does not appear yet. Or search the applications menu for those names.)

Install steps

1) Install the MSCC package

In Desktop Terminal, go to the folder with the `.deb` files:

```
cd ~/Downloads
# or:  cd /media/pi/YOUR_USB_NAME

sudo apt update
sudo apt install -y ./mscc_*_arm64.deb
```

(If the folder holds more than one mscc .deb, type the full filename instead of `mscc_*`.)

- Enter your **password** when asked.
- If it asks to continue / stop old servers → type **y** when ready.
- Install does **not** run the setup wizard for you — that comes after the log out / in.
- Upgrading from an older install: apt removes the old `mscc-portaudio`, `mscc-init`, `mscc-volume` and `mscc-firmware` packages; that is expected.

You may see a technical "Notice" about `_apt`. **If the install finished and printed "MSCC installed" / "done", ignore that notice.**

This package installs:

Piece	What it is
Servers	`ms-sdr`, `sdrcore-recv`, `sdrcore-trans` → `$HOME/mscc/`
PortAudio	MSCC's PortAudio build (Pulse + ALSA, needed for VirtualA / VirtualB) in `/usr/local/lib`; the servers are built to use it
MSCC Init	menu **MSCC Init** (`mscc-init-gui`), text version `mscc-init` (SSH / no desktop)
MSCC Volume	menu **MSCC Volume** (`mscc-volume`), levels of VirtualA / VirtualB
Firmware upload (PSoC radios)	`psoc-bootloader` (CLI) + `psoc-bootloader.gui`; menu **MSCC → Firmware Upload**
PSoC radio firmware	`/usr/share/mscc/firmware/<Radio>/` (`.cyacd` + `.hex`, the same versions as the Windows installer); Firmware Upload opens this folder
Black Pill firmware + tools	`/usr/share/mscc/firmware/black-pill/` (25 MHz board). `mscc stop; sudo stm32-app-flash /usr/share/mscc/firmware/black-pill/BlackPill-25MHz-3.16-app.bin` (USB, app) or `stm32-bootloader-flash <file.bin>` (ST-Link)
Keyer firmware + tool	Daughter-board keyer (PIC): `/usr/share/mscc/firmware/keyer/`. Stop the servers, then menu **MSCC → Keyer Firmware Update** (`keyer-flash.gui`): Load File → Program, or `keyer-flash Keyer-MCU-app-<date>.hex`. Needs Black Pill 3.16+; settings and CQ memories are kept. `Keyer-MCU-full-` = PICkit only
Start/stop	`mscc` command and desktop **MSCC Start** / **MSCC Stop**
Digi audio	`mscc-virtual-audio` creates **VirtualA** / **VirtualB**
Config seed	`$HOME/.local/mscc/` **only if empty** (never overwrites your files later)
CAT helper	tty0tty module → often `/dev/tnt0` (after build / reboot)

Quick check (optional):

```
ldd $HOME/mscc/sdrcore-recv | grep portaudio
# good: /usr/local/lib/...
# bad:  only /lib/aarch64-linux-gnu/...  → digi will often fail
```

2) Optional: MSCC Avalonia (client on the Pi)

Only if you want to operate from the Pi's own screen (not needed with the Windows client):

```
sudo apt install -y ./mscc-avalonia_*_arm64.deb
```

Menu **MSCC Avalonia**.

3) (nothing else to install)

Older guides had separate PortAudio, MSCC Init, MSCC Volume and firmware packages. They are all in `mscc` now.

4) Log out / in once (first install) — required for CAT

The package adds your user to **`dialout`**, **`plugdev`**, and **`audio`**. **Those groups do not take effect until you log out and log back in** (or reboot).

If you skip this step:

- **CAT** may fail (`/dev/tnt*` permission denied or "can't open port") even though tty0tty is loaded
- Proficio USB / sound access can also look broken

Do this once after the first `mscc` install:

- **Log out and log back in**, or reboot once.

Then check digi sinks:

```
pactl list short sinks | grep Virtual
```

If empty:

```
systemctl --user enable --now mscc-virtual-audio
# or run once:  mscc-virtual-audio
```

Role	Device name
Digital speaker (recv digi out)	**VirtualA**
Digital mic (trans digi in)	**VirtualB.monitor**
WSJT **Input** (hear radio)	**VirtualA.monitor**
WSJT **Output** (TX audio)	**VirtualB**

5) Configure this Pi (required once)

Preferred (point and click):

10. Applications menu → **MSCC** → **MSCC Init** (log out/in once if the **MSCC** category is missing).

11. Or search the menu for **MSCC Init**.

The wizard:

- Offers to **stop** servers if they are already running (safe if you refuse and exit).
- Keyer, CAT port / PTT pin.
- **Operator speaker and microphone** — pick your phones / HAT.
- **Any sample rate is OK** (48 kHz, 96 kHz, etc.).
- Do **not** pick Proficio / Multus **I/Q** for headphones or mic.
- On **Pi 4**, prefer an **I²S HAT** or **Pi headphones** over a **USB** headset (see **Operator audio hardware** above).
- Digi is **fixed**: VirtualA / VirtualB.monitor (not a menu choice).
- After save: **Yes/No** to **start the MSCC servers**.
- Sets operator hardware mixer levels toward **full open (100%)** so the **MSCC client Volume** can control loudness.

Config files live under:

```
$HOME/.local/mscc/
```

Upgrades **do not** overwrite existing config.

6) Start and stop the servers (everyday)

Preferred (point and click):

Menu (MSCC category)	What it does
MSCC Start	Starts digi sinks (best effort) + starts the three servers. **Also re-applies operator speaker/mic ALSA levels to full** (so phones stay loud after reboot).
MSCC Stop	Stops the three servers.
MSCC Status	Servers running? + config, PortAudio device match, CAT port, VirtualA/B, Proficio USB (opens a Terminal window).
MSCC Init	Configure again (stops servers first if needed).

Start and Stop show a small window saying what happened (for example "Servers started", "Servers already running" or "Servers were not running"). It closes by itself after 5 seconds; a warning or error stays until you press OK.

CLI alternative:

```
mscc start
mscc status    # servers + full install check (config / audio / CAT)
mscc stop
mscc restart
```

`mscc status` also checks that operator and digi names in `$HOME/.local/mscc` match PortAudio devices (needs `python3-pyaudio` for the device list).

Then use the **MSCC client on your PC** to operate the radio (Pi IP / hostname as configured).

Operator phones vs digi (what to expect)

Phones (operator path)

- Proficio radio I/Q is always **96 kHz** on the wire.
- Your operator device can be **96 kHz or not** (e.g. 48 kHz). The servers handle the difference.
- On **Pi 4**, best results with an **I²S HAT** or **Pi jack** — not a second USB sound card (see **Operator audio hardware**).
- **Volume after reboot:** run **MSCC Start** (or `mscc start`). Levels are re-applied automatically.
- If still quiet: re-run **MSCC Init**, or raise levels once with the desktop volume control / `alsamixer` for that sound card.
- Day-to-day loudness: use the **MSCC client Volume** control (not only the Pi desktop mixer forever).

Digi (WSJT etc.)

Typical WSJT-X on the Pi:

WSJT setting	Value
Input	**VirtualA.monitor**
Output	**VirtualB**
CAT	As set in MSCC Init (often `/dev/tnt0`)

Digi uses **VirtualA / VirtualB** (software). That does **not** add a second USB sound device, so it is a good fit for **Pi 4** as well as Pi 5.

Digi levels use **Pulse/PipeWire**, not the same as phone `alsamixer`.

If WSJT's receive bar is weak while the MSCC client is already loud:

```
pactl set-sink-volume VirtualA 100%
pactl set-sink-mute VirtualA 0
```

(That raises the digi path into WSJT. Then trim with MSCC client Volume if needed.)

WSJT-X and CPU (especially Pi 4): MSCC already uses real CPU for the radio. FT8 with **many decoder threads** can load the Pi heavily. If the desktop feels sluggish or decodes fall behind, lower WSJT's thread count (or use single-thread decode). A **Pi 5** has more headroom for MSCC + multi-thread FT8 together.

Virtual sinks should return at **desktop login** via the `mscc-virtual-audio` user service. If digi vanishes after reboot:

```
systemctl --user enable --now mscc-virtual-audio
```

Did it work?

```
mscc status
ls /dev/tnt0 /dev/tnt1
pactl list short sinks | grep Virtual
ldd $HOME/mscc/sdrcore-recv | grep portaudio
```

Check	Good sign
`mscc status`	`sdrcore-recv`, `sdrcore-trans`, and `ms-sdr` show

	running
`/dev/tnt0`	Exists (CAT / PTT when tty0tty built)
`pactl … Virtual`	**VirtualA** / **VirtualB** present
`ldd … portaudio`	**`/usr/local/lib`**
PC client	Connects to this Pi

Logs (if something fails):

- Servers: `$HOME/.local/mscc/sdrcore-recv.log`, `sdrcore-trans.log`, `ms-sdr.log`
- Start script text: `$HOME/mscc/logs/`

Useful digi / audio lines:

```
grep -E "VirtualA|DUAL STREAM|ALSA card|portaudio|FAILED" ~/.local/mscc/sdrcore-recv.log | tail -30
```

Everyday use (after first install)

12. Power on Pi and radio.
13. Log into the Pi desktop (so Pulse/PipeWire and Virtual* can start).
14. Menu → **MSCC** → **MSCC Start** (or Terminal: `mscc start`).
15. Use **MSCC on the PC** as usual.
16. When done: **MSCC Stop** (optional if you leave the Pi running).

Re-run **MSCC Init** only if you change phones, HAT, or CAT wiring.

If something fails

Symptom	**What to try**		
`mscc: command not found`	New Desktop Terminal, or log out/in		
No menu entries	Confirm packages installed; look under **MSCC**; log out/in		
`mscc-init-gui` missing	Install `mscc` 1.0.79 or later (MSCC Init is in it); needs a desktop		
No `/dev/tnt0`	Reboot once; install headers: `sudo apt install -y linux-headers-$(uname -r)`. After a kernel update (1.0.74+ rebuilds it by itself, see `/var/log/mscc-tty0tty.log`), or by hand: `sudo /usr/share/mscc/bin/mscc-tty0tty-kernel $(uname -r)` then `sudo modprobe tty0tty`		

No VirtualA/B	`mscc-virtual-audio` or `systemctl --user enable --now mscc-virtual-audio`		
Digi devices not listed in init / WSJT	Check `ldd $HOME/mscc/sdrcore-recv` → `/usr/local`; reinstall `mscc`		
Wrong PortAudio (`ldd` shows only `/lib/...`)	Reinstall `mscc` (it carries PortAudio since 1.0.79)		
Quiet phones after reboot	**MSCC Start** (re-applies levels); re-run **MSCC Init** if needed		
Quiet digi / WSJT RX weak	`pactl set-sink-volume VirtualA 100%` ; check WSJT Input = VirtualA.monitor		
Audio wrong device	Re-run **MSCC Init**. Digi stays VirtualA / VirtualB.monitor		
Spectrum / phones freeze; `mscc status` still **running**	Often USB audio dropped under load. On **Pi 4**, avoid a **USB headset + Proficio**. Use an **I²S HAT** or digi path. Then `mscc stop` / `mscc start` and reconnect the client. Check: `dmesg -T |	grep -iE 'disconnect	usb'`
WSJT makes the Pi very busy / late decodes	Lower WSJT decoder threads on **Pi 4**; prefer digi VirtualA/B; consider **Pi 5** for heavy multi-thread FT8 + MSCC		
"Permission denied" on `/dev/tnt*` or CAT won't open	**Log out/in or reboot** so `dialout` applies; check `groups` and `ls -l /dev/tnt*`		
Servers won't start from menu	Terminal: `mscc start` and read the error text		
Client won't connect	Pi and PC same network; firewall; correct Pi IP; servers running (`mscc status`)		
AudioInjector HAT silent after first install	Reboot once if the package added the audio overlay		

This stack is only for Raspberry Pi OS 64-bit on Pi 4/5. Other Linux systems are not supported.

What the three servers are (optional reading)

Program	Job

sdrcore-recv	Receive: radio I/Q in, audio to phones and digi out
sdrcore-trans	Transmit: mic / digi in, radio I/Q out
ms-sdr	Control hub: talks to Proficio and to the PC client

You do not start them individually for normal use — use **MSCC Start** / `mscc start`.

One-line cheat sheet

```
Desktop Terminal (order matters):
  1. sudo apt install -y ./mscc_*_arm64.deb        (servers, PortAudio, Init, Volume,
firmware)
  Optional: sudo apt install -y ./mscc-avalonia_*_arm64.deb
  (log out/in if first time)
  Optional checks:
     ldd $HOME/mscc/sdrcore-recv | grep portaudio
     pactl list short sinks | grep Virtual

Menu → MSCC:
  MSCC Init  →  configure once  →  Yes to start (or MSCC Start later)
  MSCC Start / MSCC Stop / MSCC Status day to day

PC: run MSCC client → connect to Pi

WSJT (on Pi): Input = VirtualA.monitor   Output = VirtualB

Pi 4 operator audio: I²S HAT or Pi jack preferred (not USB headset + Proficio)
Pi 4 + FT8: keep WSJT thread count modest
```

That's it.

For developers / handoff

See **README.md** (package build) and **resume.md** (status, audio/pan contracts, open items). Do not put long engineering notes in this operator card — keep this file short and one path.